UNIVERSIDAD / ASIGNATURA: [completar]

Sistema Web de Gestión de Inventario

Entrega: Diagramas lógicos, prototipado y esqueleto del código

---

Integrantes

[Nombre del integrante 1]

[Nombre del integrante 2]

[Nombre del integrante 3]

Repositorio:

https://github.com/<usuario>/sistema-inventario

Stack: Python - Django (MVT) - MySQL - HTML5 - CSS3 - Bootstrap - JavaScript - Git/GitHub

Fecha: Octubre de 2026

Contenido

1. Diagramas lógicos (modelado del sistema): flujo, casos de uso, clases y arquitectura
2. Prototipado: árbol de navegación y wireframes de la interfaz gráfica
3. Esqueleto del código y configuración del entorno: repositorio, carpetas, stubs y estructuras de datos
4. Enlace al repositorio

Alcance del proyecto

Sistema web académico y deliberadamente simple para gestionar productos y categorías de un inventario: registrar, consultar, buscar, actualizar y eliminar productos, validar la información, controlar el stock de forma básica y administrar categorías. Se desarrolla con Python y Django (arquitectura MVT) sobre MySQL. Quedan fuera de alcance la facturación, los pagos, los reportes complejos y las tecnologías ajenas al stack indicado.

1. Diagramas lógicos (modelado del sistema)

1.1 Diagrama de flujo - Gestión de Productos

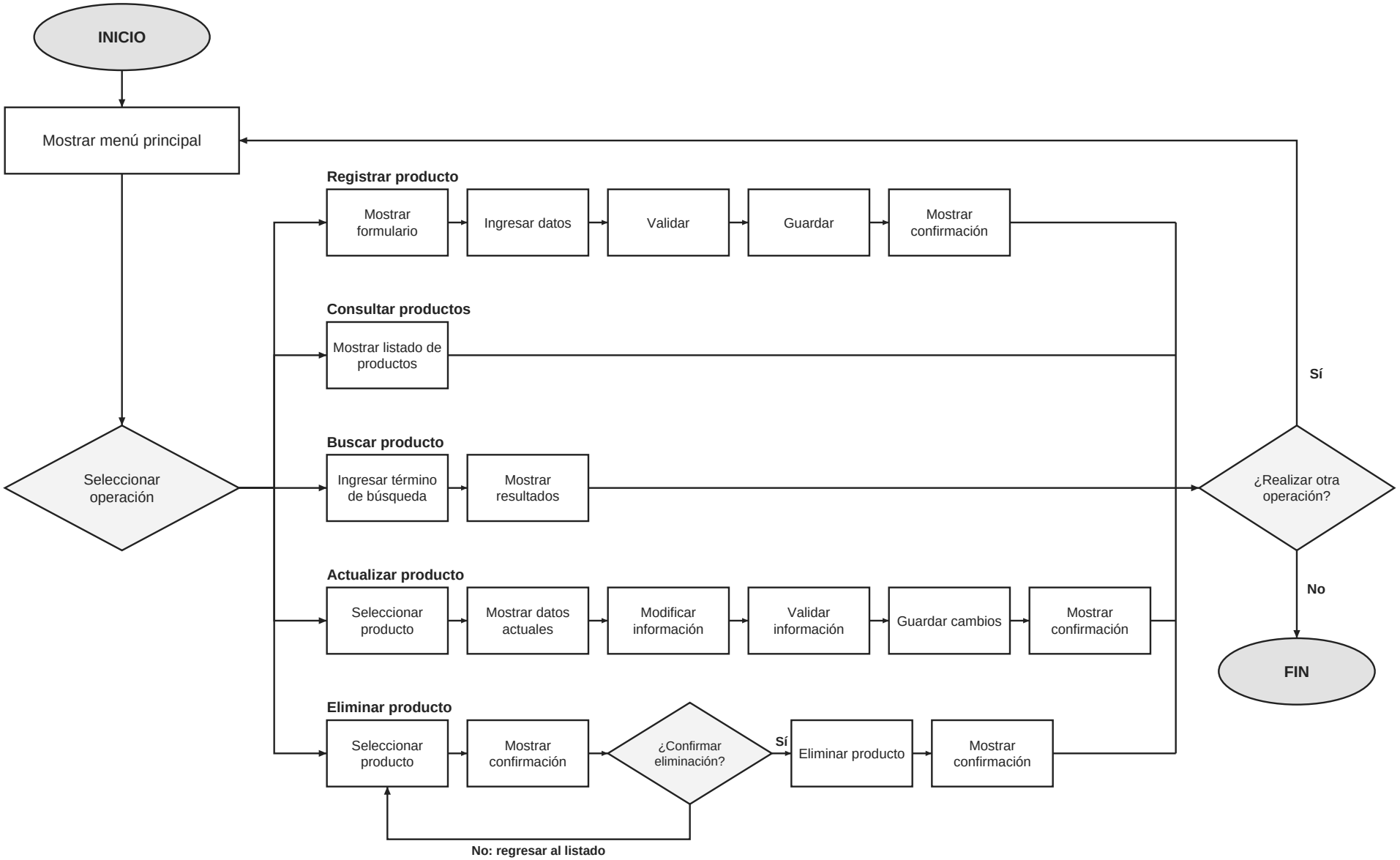


Figura 1. Flujo general: menú, selección de operación y rutas de registrar, consultar, buscar, actualizar y eliminar.

1.2 Diagrama de flujo - Registro de Producto

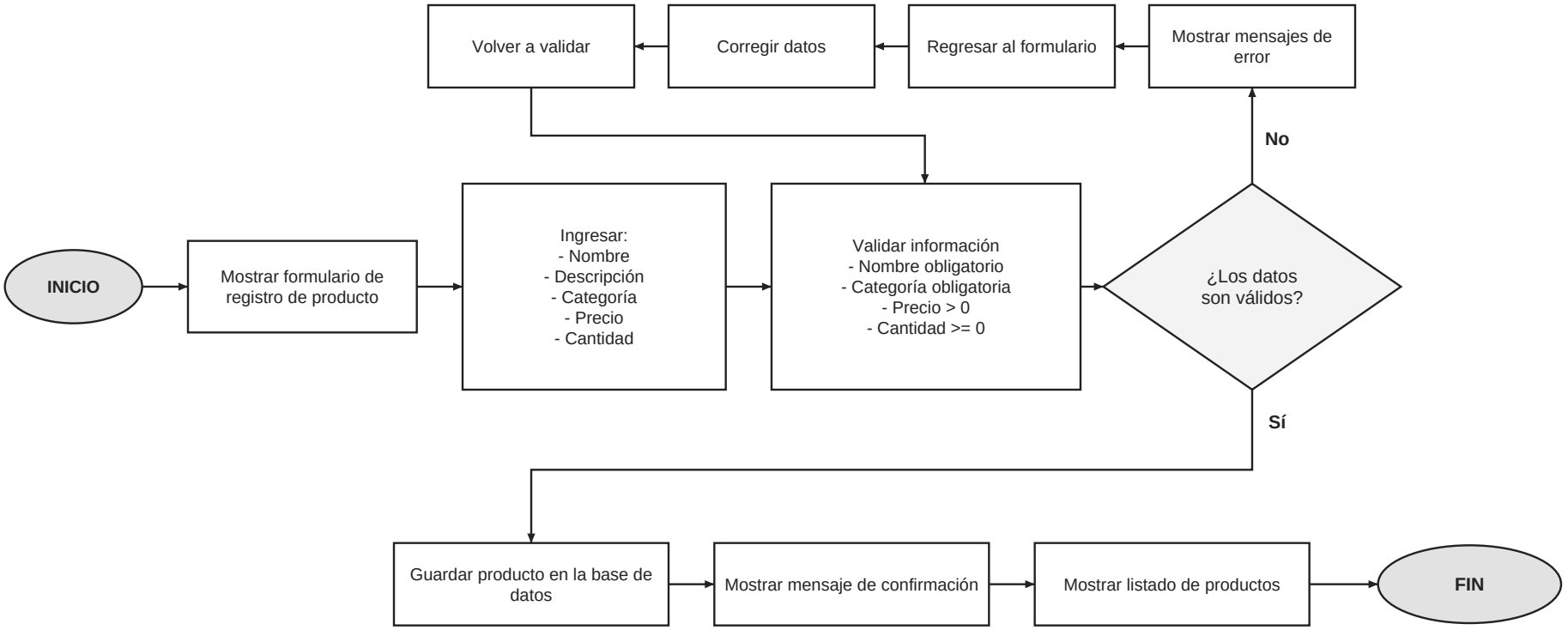


Figura 2. Proceso crítico de validación de entrada con ciclo de corrección. Validaciones: nombre y categoría obligatorios, precio mayor que cero, cantidad mayor o igual a cero.

1.3 Diagrama de casos de uso (UML)

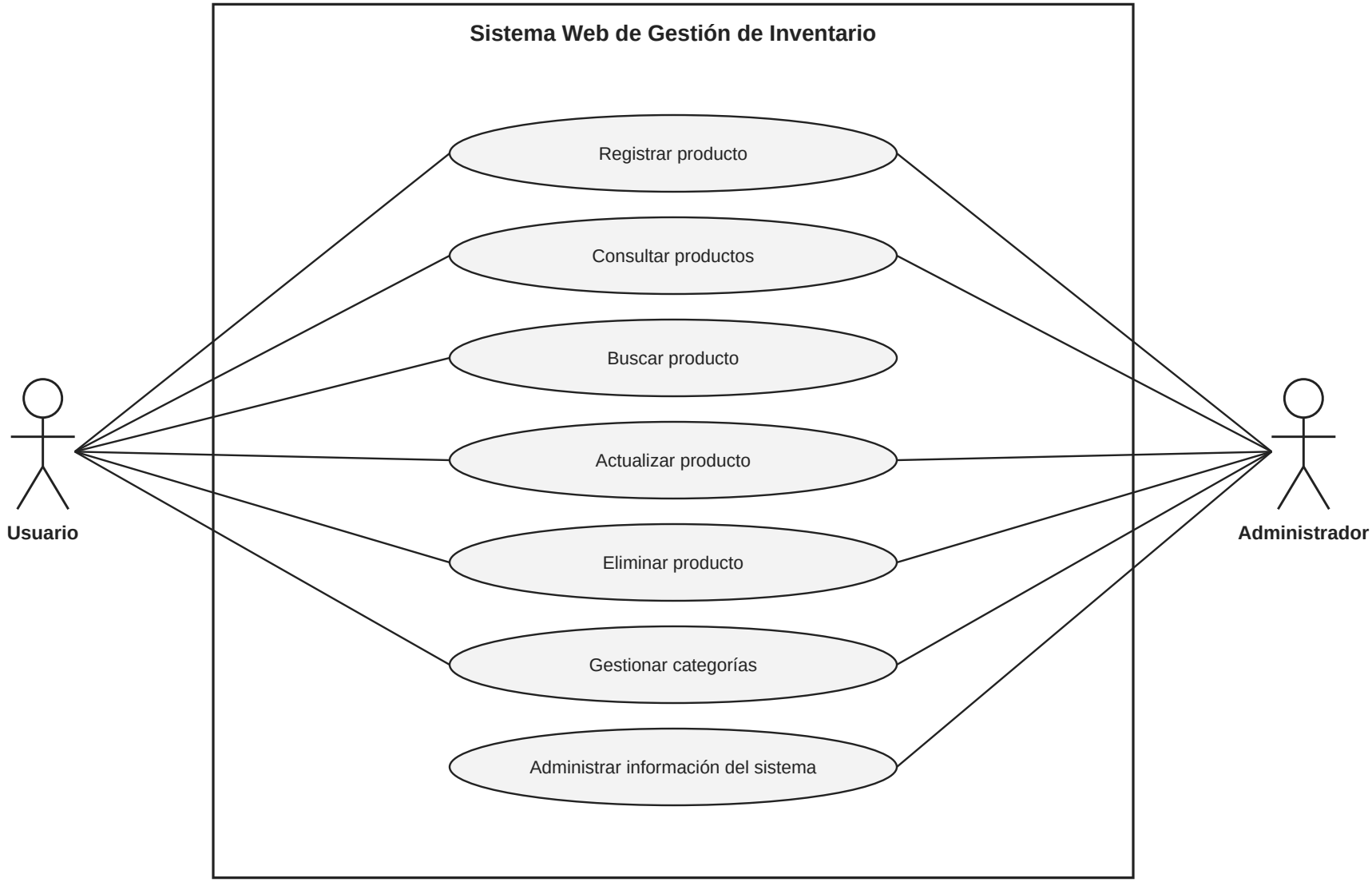


Figura 3. Actores fuera del límite del sistema; casos de uso como óvalos dentro del límite.

Detalle de actores y acciones

Actor	Acciones que puede ejecutar
Usuario	Registrar producto, consultar productos, buscar producto, actualizar producto, eliminar producto y gestionar categorías.
Administrador	Registrar producto, consultar productos, actualizar producto, eliminar producto, gestionar categorías y administrar información del sistema (panel administrativo de Django).

Límite del sistema: todo lo que ocurre dentro del rectángulo "Sistema Web de Gestión de Inventario". Las relaciones son asociaciones simples entre actor y caso de uso; no se usan include ni extend. El Usuario no accede a "Administrar información del sistema" y el Administrador no tiene asociada la búsqueda en este modelo.

1.4 Diagrama de clases (UML)

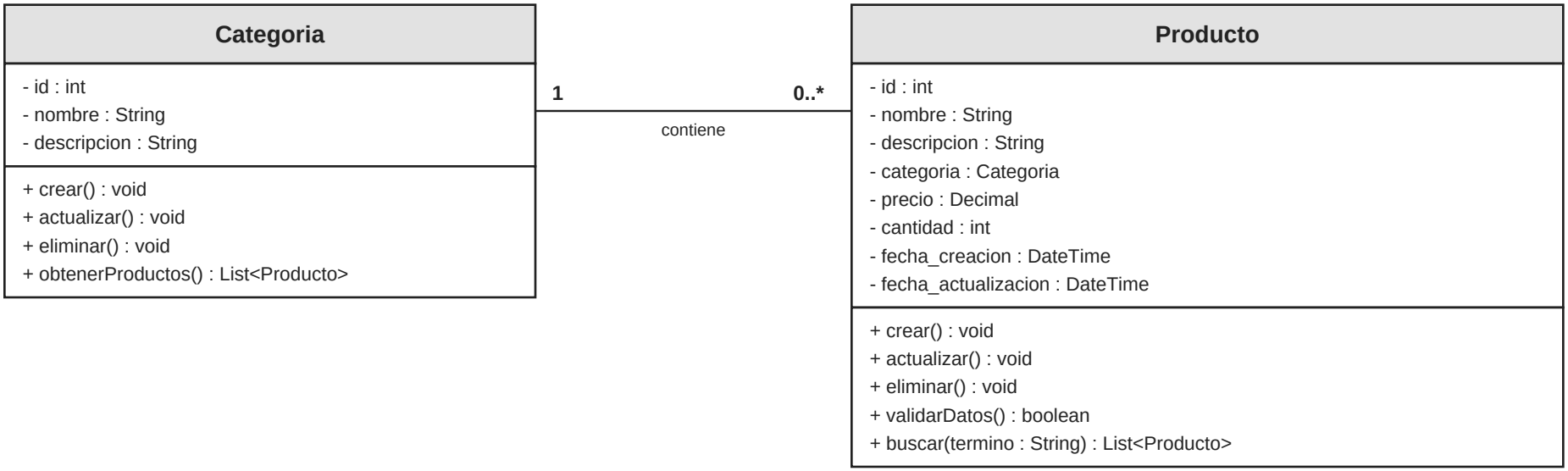


Figura 4. Clases Categoria y Producto con atributos tipados (-), métodos con parámetros y retornos (+) y cardinalidad 1 a 0..*.

Relaciones: asociación Categoria 1 --- 0..* Producto (una categoría tiene cero o muchos productos; cada producto pertenece a una sola categoría). El modelo es intencionalmente simple, por lo que no requiere herencia ni composición.

1.5 Arquitectura MVT de Django

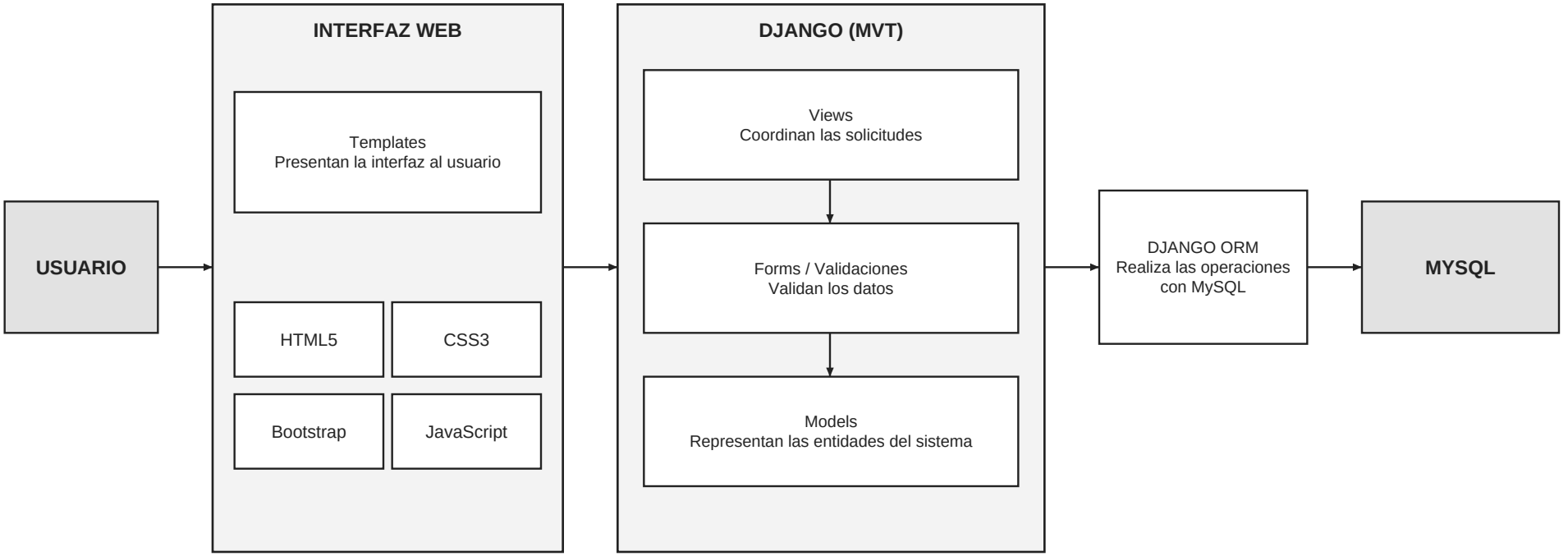


Figura 5. Usuario - Interfaz web - Django (Views, Forms/Validaciones, Models) - Django ORM - MySQL.

2. Prototipado (diseño de interfaz y experiencia)

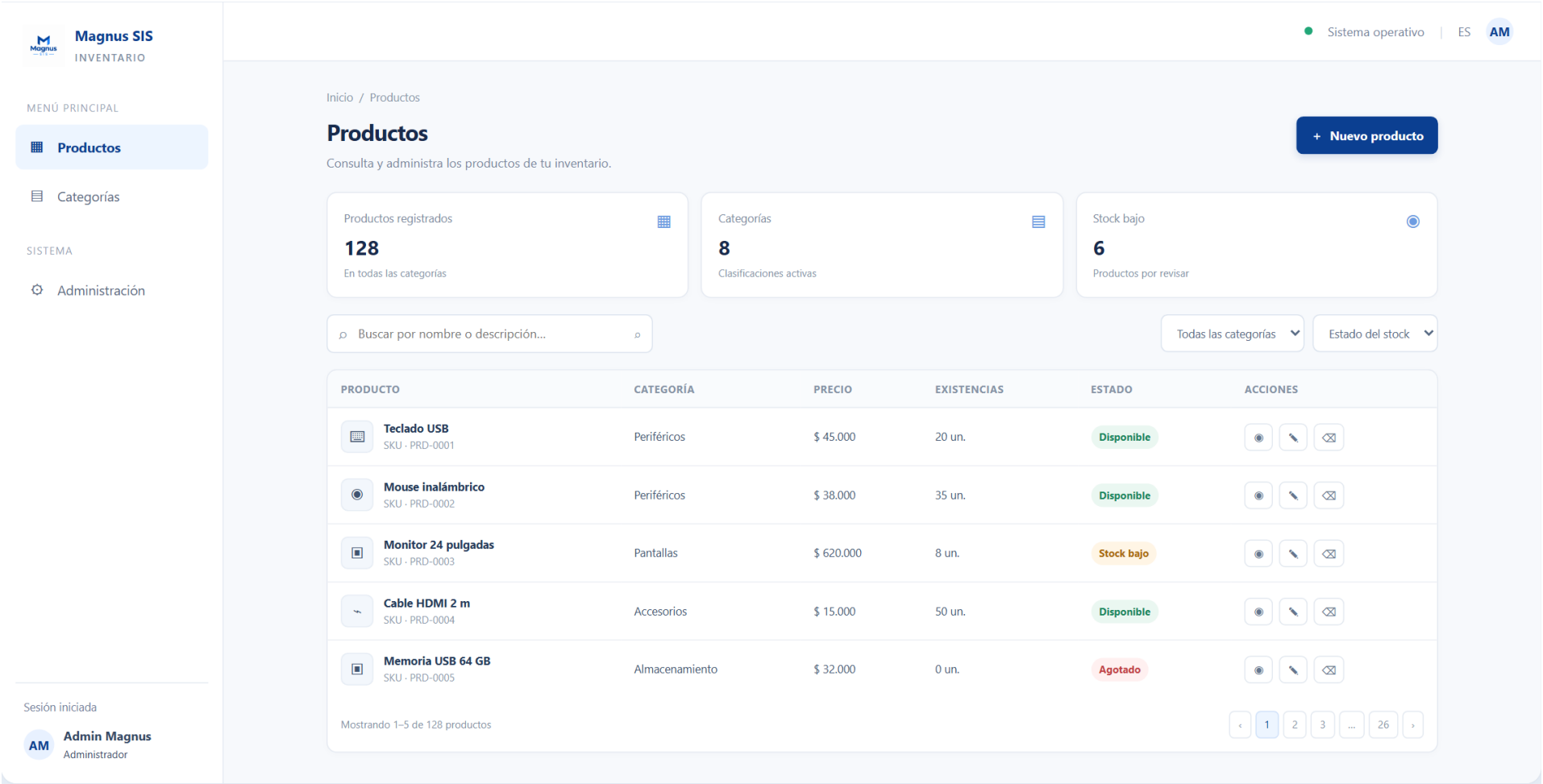
El proyecto es una aplicación web, por lo que el prototipo se presenta con wireframes de interfaz gráfica; no aplica una interfaz por consola (CLI). A continuación se muestra el árbol de navegación que guía el diseño de las pantallas.

```
Listado de productos (inicio)  /
|-- Buscar producto (termino en el listado)       /buscar/
|-- Nuevo producto (formulario + validacion)      /productos/nuevo/
|-- Detalle del producto                          /productos/<id>/
|     |-- Editar producto (formulario)            /productos/<id>/editar/
|     `-- Eliminar producto (confirmacion)        /productos/<id>/eliminar/
|-- Categorias (listado)                          /categorias/
`-- Panel administrativo de Django                /admin/
```

2.1 Mockup · Inventario de productos

Listado principal con búsqueda, filtros, estado de existencias y accesos a las operaciones CRUD.

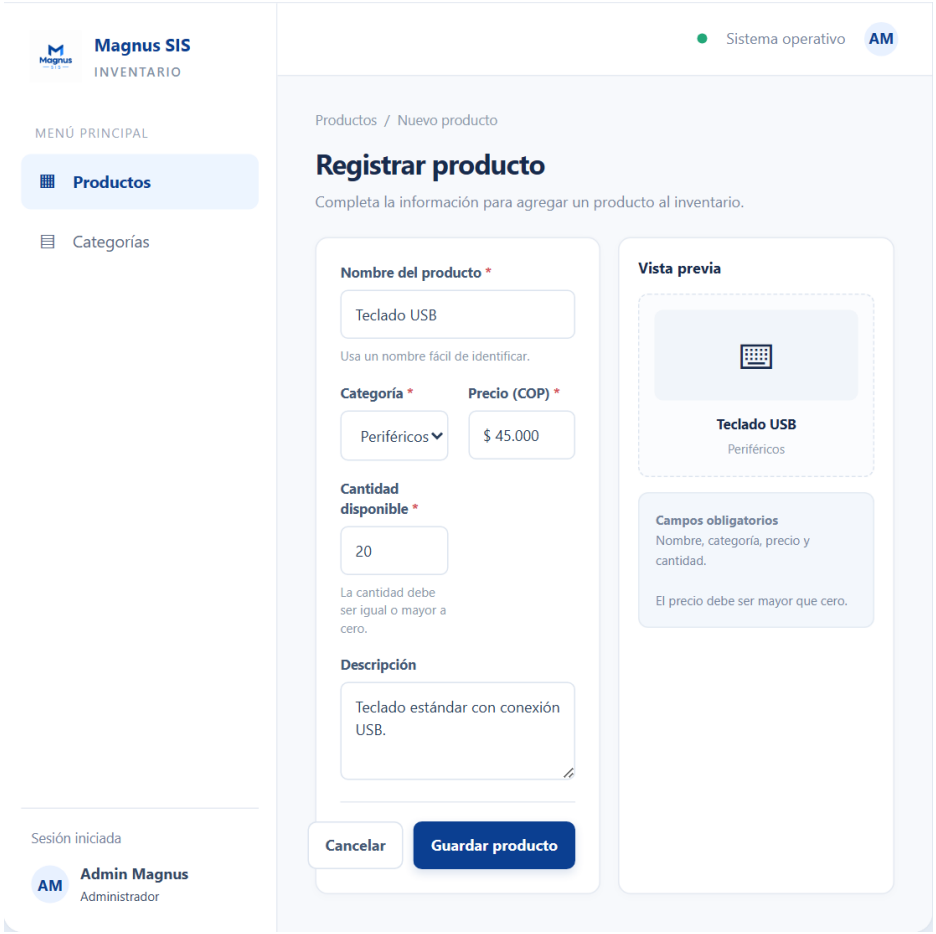
Listado y búsqueda



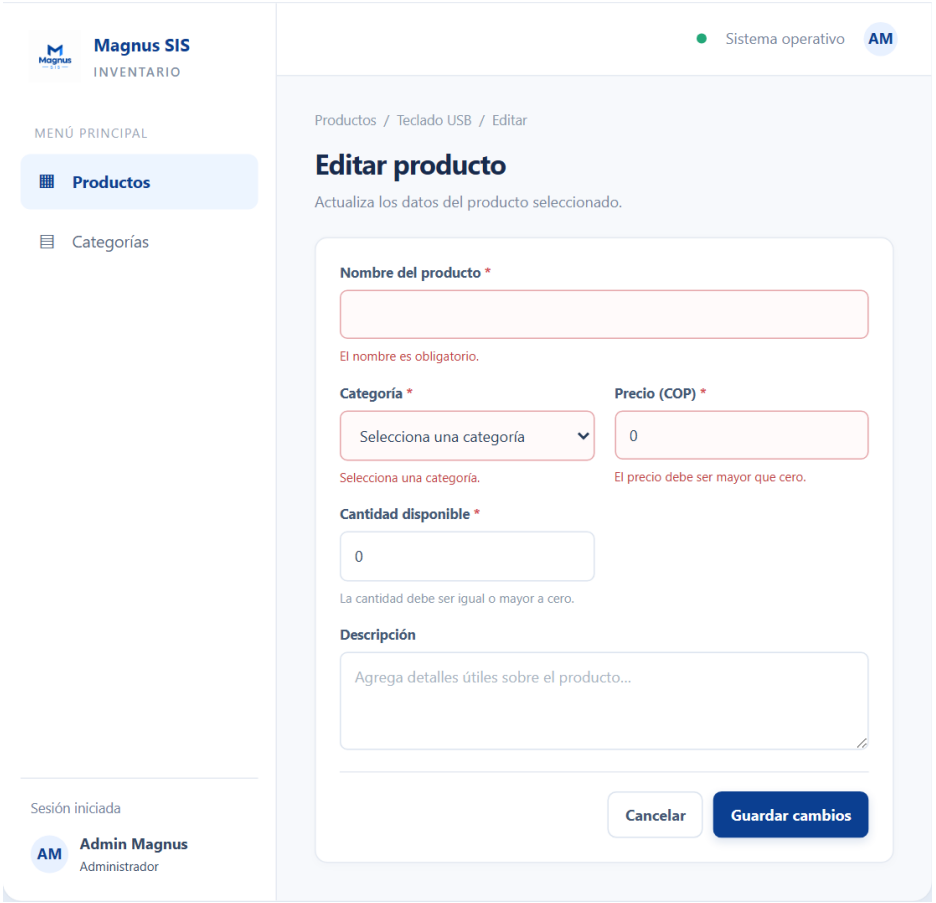
2.2 Mockups · Registro y edición

Formularios de producto con los campos y reglas de validación definidos para el sistema.

Registrar producto



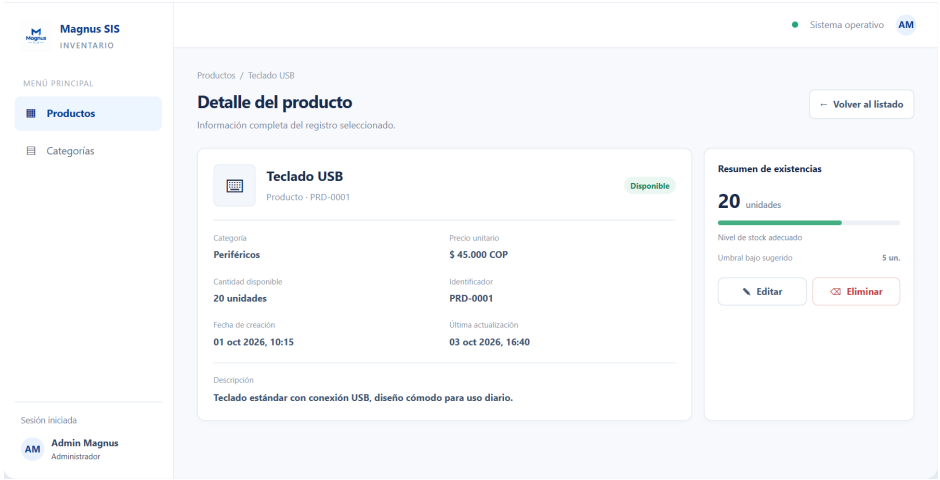
Editar · mensajes de validación



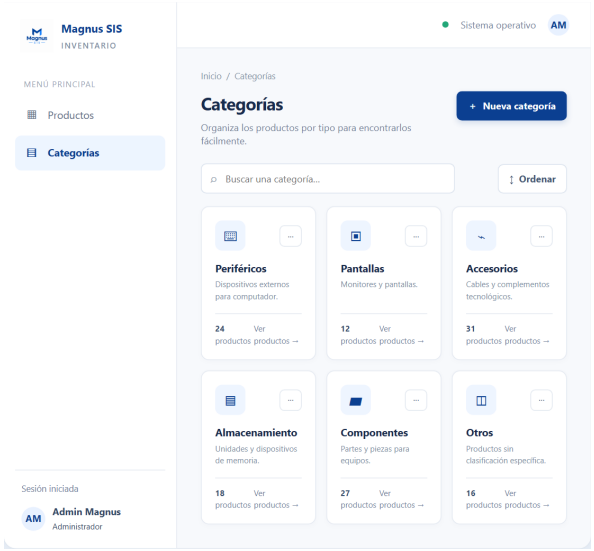
2.3 Mockups · Detalle, categorías y eliminación

Consulta de producto, administración de categorías y confirmación antes de eliminar.

Detalle



Categorías



Confirmación de eliminación



3. Esqueleto del código y configuración del entorno

3.1 Repositorio y control de versiones

Repositorio en GitHub con rama principal **main**, archivo **.gitignore** para Python/Django y **README.md** inicial con título, integrantes y descripción. URL: **https://github.com/<usuario>/sistema-inventario**

3.2 Arquitectura de carpetas

```
sistema-inventario/
|-- .gitignore
|-- README.md
|-- requirements.txt
|-- manage.py
|-- config/                    (configuracion del proyecto Django)
|   |-- settings.py  urls.py  wsgi.py
|-- productos/                 (aplicacion: codigo fuente MVT)
|   |-- models.py  forms.py  views.py  urls.py  admin.py  tests.py
|   |-- migrations/0001_initial.py
|   `-- templates/productos/   base, lista, detalle, form, confirmar_eliminar (.html)
|-- docs/                      (documentacion y diagramas)
`-- assets/data/               (estructura de datos y recursos)
```

Verificación: **python manage.py check** sin errores y 2 pruebas de humo en OK (el listado y el formulario responden 200). Sin lógica de negocio.

3.3 Clases y funciones stub

productos/models.py

```
"""Modelos del sistema (stubs: sin lógica de negocio)."""
from django.db import models


class Categoria(models.Model):
    nombre = models.CharField(max_length=100)
    descripcion = models.TextField(blank=True)

    def __str__(self) -> str:
        return self.nombre

    def crear(self) -> None:
        pass

    def actualizar(self) -> None:
        pass

    def eliminar(self) -> None:
        pass

    def obtener_productos(self) -> list:  # UML: obtenerProductos()
        return []


class Producto(models.Model):
    nombre = models.CharField(max_length=150)
    descripcion = models.TextField(blank=True)
    categoria = models.ForeignKey(
        Categoria, on_delete=models.PROTECT, related_name="productos"
    )
    precio = models.DecimalField(max_digits=10, decimal_places=2)
    cantidad = models.PositiveIntegerField(default=0)
    fecha_creacion = models.DateTimeField(auto_now_add=True)
    fecha_actualizacion = models.DateTimeField(auto_now=True)

    def __str__(self) -> str:
        return self.nombre

    def crear(self) -> None:
        pass

    def actualizar(self) -> None:
        pass

    def eliminar(self) -> None:
        pass

    def validar_datos(self) -> bool:  # UML: validarDatos()
        return False

    @staticmethod
    def buscar(termino: str) -> list:
        return []
```

productos/forms.py

```
"""Formularios y validaciones (stubs)."""
from django import forms

from .models import Categoria, Producto


class ProductoForm(forms.ModelForm):
    class Meta:
        model = Producto
        fields = ["nombre", "descripcion", "categoria", "precio", "cantidad"]

    def clean(self) -> dict:
        # Pendiente: nombre y categoría obligatorios, precio > 0, cantidad >= 0
        return super().clean()


class CategoriaForm(forms.ModelForm):
    class Meta:
        model = Categoria
        fields = ["nombre", "descripcion"]
```

productos/views.py

```
"""Vistas del sistema (stubs: devuelven plantillas vacías)."""
from django.http import HttpRequest, HttpResponse
from django.shortcuts import render


def lista_productos(request: HttpRequest) -> HttpResponse:
    return render(request, "productos/lista.html", {})


def buscar_producto(request: HttpRequest) -> HttpResponse:
    return render(request, "productos/lista.html", {})


def detalle_producto(request: HttpRequest, pk: int) -> HttpResponse:
    return render(request, "productos/detalle.html", {})


def crear_producto(request: HttpRequest) -> HttpResponse:
    return render(request, "productos/form.html", {})


def editar_producto(request: HttpRequest, pk: int) -> HttpResponse:
    return render(request, "productos/form.html", {})


def eliminar_producto(request: HttpRequest, pk: int) -> HttpResponse:
    return render(request, "productos/confirmar_eliminar.html", {})


def lista_categorias(request: HttpRequest) -> HttpResponse:
    return HttpResponse("Categorías: pendiente de implementar")
```

productos/urls.py

```
"""Rutas de la aplicación productos."""
from django.urls import path

from . import views

app_name = "productos"

urlpatterns = [
    path("", views.lista_productos, name="lista"),
    path("buscar/", views.buscar_producto, name="buscar"),
    path("productos/nuevo/", views.crear_producto, name="crear"),
    path("productos/<int:pk>/", views.detalle_producto, name="detalle"),
    path("productos/<int:pk>/editar/", views.editar_producto, name="editar"),
    path("productos/<int:pk>/eliminar/", views.eliminar_producto, name="eliminar"),
    path("categorias/", views.lista_categorias, name="categorias"),
]
```

productos/tests.py

```
from django.test import TestCase
from django.urls import reverse


class HumoTests(TestCase):
    def test_lista_responde(self):
        self.assertEqual(self.client.get(reverse("productos:lista")).status_code, 200)

    def test_formulario_responde(self):
        self.assertEqual(self.client.get(reverse("productos:crear")).status_code, 200)
```

3.4 Estructuras de datos y configuración

Persistencia en MySQL mediante el ORM de Django (SQLite por defecto para desarrollo local).

```
# Estructura de datos (MySQL / Django ORM)

categoria(id PK, nombre VARCHAR(100) NOT NULL, descripcion TEXT)
producto(id PK, nombre VARCHAR(150) NOT NULL, descripcion TEXT,
         categoria_id FK -> categoria.id NOT NULL,
         precio DECIMAL(10,2) NOT NULL, cantidad INT UNSIGNED DEFAULT 0,
         fecha_creacion DATETIME, fecha_actualizacion DATETIME)
```

.gitignore

```
# Python
__pycache__/
*.py[cod]
*.egg-info/
.coverage
htmlcov/

# Entornos virtuales y secretos
venv/
.venv/
env/
.env

# Django
db.sqlite3
*.sqlite3
*.log
staticfiles/
media/

# IDE / Sistema operativo
.idea/
.vscode/
.DS_Store
Thumbs.db
```

requirements.txt

```
Django>=5.0
# Para usar MySQL en lugar de SQLite, instalar tambien:
# mysqlclient>=2.2
```

README.md

````
# Sistema Web de Gestión de Inventario

**Integrantes:**
- [Nombre del integrante 1]
- [Nombre del integrante 2]
- [Nombre del integrante 3]

## Descripción
Aplicación web académica para gestionar productos y categorías de un inventario
básico: registrar, consultar, buscar, actualizar y eliminar productos, con
validaciones y control básico de stock.

## Tecnologías
Python, Django (arquitectura MVT), MySQL, HTML5, CSS3, Bootstrap, JavaScript, Git y GitHub.

## Estado
Fase de esqueleto: estructura, clases y funciones *stub*. Aún sin lógica de negocio.

## Ejecución
```bash
python -m venv .venv
source .venv/bin/activate        # Windows: .venv\Scripts\activate
pip install -r requirements.txt
python manage.py migrate
python manage.py runserver
```
Por defecto usa SQLite. Para MySQL definir `DB_ENGINE=mysql`, `DB_NAME`,
`DB_USER`, `DB_PASSWORD`, `DB_HOST` y `DB_PORT`.

## Estructura
- `config/`    configuración del proyecto Django
- `productos/` aplicación (models, forms, views, urls, templates)
- `docs/`      documentación y diagramas
- `assets/`    recursos y estructuras de datos de apoyo
````

4. Enlace al repositorio

https://github.com/<usuario>/sistema-inventario